- 학습 데이터를 더 넣으면 일반적으로 예측의 불확실성이 줄어들고 정답에 가까워질 수 있음!
- 단, 조건이 있음.
- → 학습 데이터가 실제 예측 대상과 같은 분포에서 왔고, 그 분포를 대표할 수 있어야 함.
- → 모델의 표현 능력과 학습 방법도 충분히 뒷받침되어야 함.
- 이런 조건이 만족되면 데이터가 늘어날수록 일반화 성능이 좋아지고 예측의 불확실성이 줄어드는 경향이 있음.
- 하지만 데이터만 무조건 늘린다고 성능 향상이 보장되는 것은 아님!
- → 데이터의 대표성, 모델의 귀납적 편향, 학습 과정도 중요함.
- 적은 데이터 + 귀납적 가설이 맞는 모델
- → 적은 데이터로도 더 좋은 예측이 가능하다
- ≡ 데이터를 더 넣거나 or 더 좋은 아키텍처를 만들던가
- EX
- SIREN(Sinusoidal Representation Networks)
- → 사인 함수(sin)를 활성화 함수로 사용하는 신경망!
- → 암시적 신경 표현(Implicit Neural Representation)의 한 종류임.
- → 좌표를 입력받아 해당 좌표의 값을 출력하는 연속 함수를 학습함. (좌표 → 값)
- EX
- → 이미지: (x, y) 좌표 → 해당 위치의 RGB 값
- → 오디오: 시간 t → 해당 시점의 신호 값
- → 3D 형상: (x, y, z) 좌표 → 해당 위치의 형상 관련 값
- 즉, 이미지 전용 모델이 아니라 이미지·오디오·3D 형상 등 다양한 연속 신호를 함수로 fitting하는 모델!

- CNN
- 이미지를 인식하려면?

1. 이미지 전체를 한 번에 판단하는 대신, 이미지의 작은 영역(Patch)에 나타나는 특징부터 살펴볼 수 있음!
   → 이미지의 일부 영역에서 Edge(경계), Texture(질감) 같은 지역적 특징을 추출함.
   → 이런 지역적 특징들을 여러 단계에 걸쳐 결합하면 더 복잡한 형태나 객체의 특징을 파악할 수 있음.
   실제 CNN은 각 패치를 독립적으로 Classifier에 넣어 분류하는 구조는 아님!
   → Convolution Filter를 이미지의 여러 위치에 반복 적용하여 지역적 특징을 추출함.
   → 여러 Convolution Layer를 쌓으면서 작은 지역 특징을 더 넓은 영역의 특징으로 계층적으로 결합함.
   → 최종적으로 추출된 특징을 이용해 이미지 전체의 클래스를 예측할 수 있음. (Image Classification의 경우)
   즉, 패치 단위로 지역 정보를 살펴본다는 아이디어에서 출발하지만, CNN의 핵심은 Convolution을 통한 지역적 특징 추출 + 계층적 특징 학습에 있음!

- 패치를 더 작게 나누면 화질이 높아질까?
- → 아님! 원본 이미지를 더 작은 패치로 나눈다고 이미지의 해상도나 정보량 자체가 증가하는 것은 아님.

- → 다만 더 작은 영역을 기준으로 지역적 특징을 살펴볼 수 있어 세밀한 위치 정보를 다루는 데 도움이 될 수 있음.
- 하지만 패치가 너무 작으면?
- → 하나의 패치에 포함되는 정보와 주변 문맥(Context)이 줄어들기 때문에, 그 패치만으로 무엇인지 식별하기 어려워질 수 있음.
- 반대로 패치가 크면?
- → 더 넓은 문맥을 한 번에 볼 수 있지만, 작은 영역의 세부 특징을 구분하는 데 불리할 수도 있음.
- 즉, 패치 크기는 화질을 높이는 수단이 아니라, 지역적 세부 정보와 넓은 문맥 사이의 균형을 결정하는 요소임!
- CNN에서는 여러 Convolution Layer를 쌓아 작은 지역 특징을 결합하면서 점차 넓은 문맥까지 학습할 수 있음.

1. Convolution

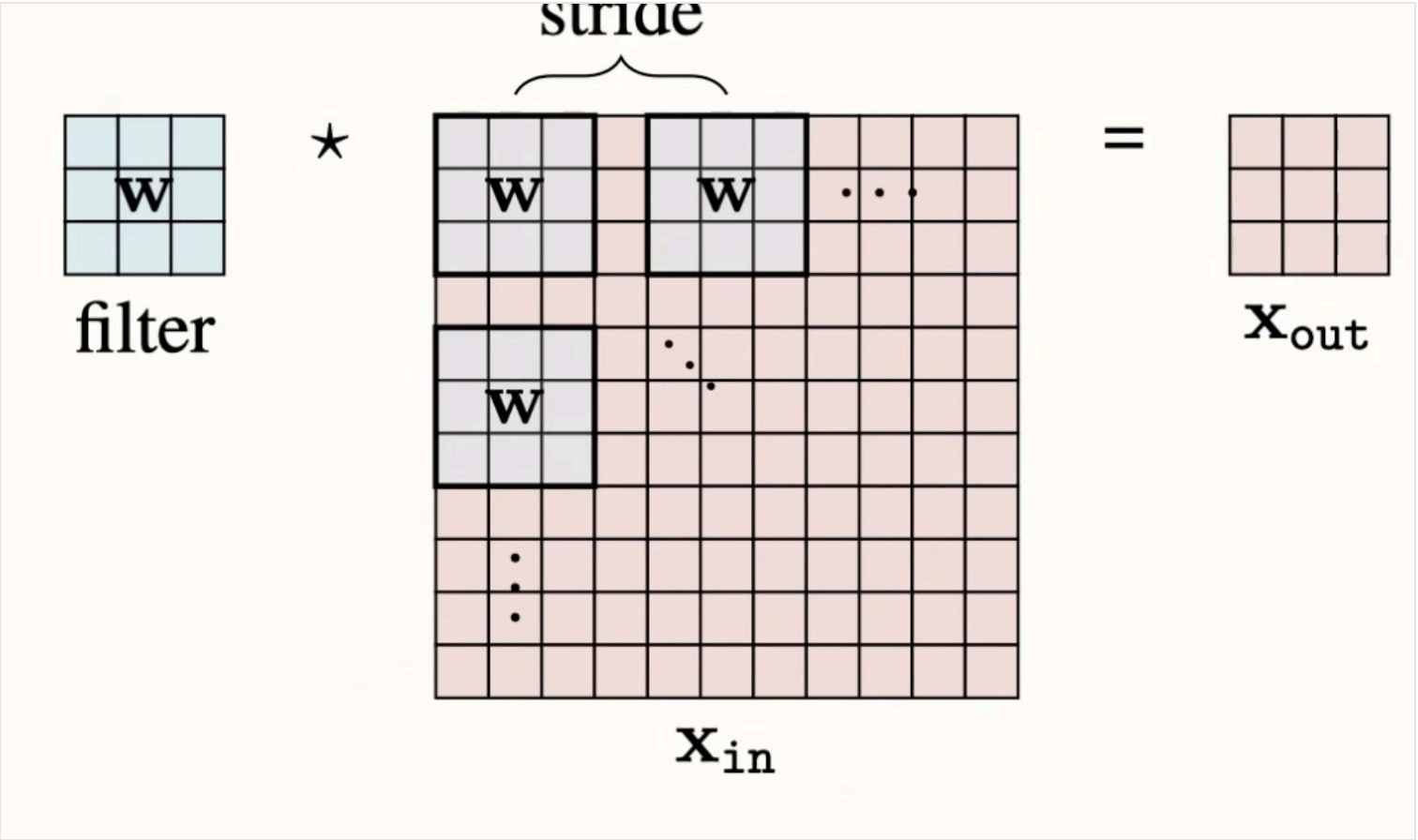
- 우리가 아는 cnn에서의 filter를 쓰면 됨
- 합성곱(Convolution)도 행렬 곱으로 표현할 수 있음!
- 입력을 펼치면 Toeplitz 구조의 가중치 행렬로 나타낼 수 있음. (2D·다채널에서는 block-Toeplitz 구조)
- 하지만 일반적인 Fully Connected Layer와 같지는 않음!
- → 지역성(Locality): 각 출력이 입력의 일부 영역에만 연결됨. (희소 연결)
- → 가중치 공유(Weight Sharing): 같은 필터를 여러 위치에 반복 적용함.
- 즉, 합성곱은 지역성과 가중치 공유라는 제약을 가진 Fully Connected Layer의 특수한 형태(부분집합)라고 볼 수 있음.
- → Convolution은 지역 연결(Local Connectivity) + 가중치 공유(Weight Sharing)를 사용하는 구조!
- Locally Connected Layer도 주변의 일부 영역에만 연결되지만, 위치마다 서로 다른 가중치를 사용함. (가중치 공유 X)
- 반면 Convolution Layer는 같은 필터의 가중치를 여러 위치에서 공유함.
- 즉, 둘 다 지역 연결을 사용하지만 가중치 공유 여부가 핵심 차이!

- color?
- Color?
- 이미지가 RGB 색상 정보를 가진다면?
- → 입력 이미지의 형태: $H \times W \times 3$
- → H: Height(높이)
- → W: Width(너비)
- → 3: RGB 색상 채널 수
- Convolution은 공간적인 특징을 추출하면서 채널 수를 변경할 수 있음!
- EX
- → Input: $H \times W \times 3$ (RGB 이미지)
- → Convolution → Output: $H' \times W' \times C_{out}$
- 여기서 C_{out}은 출력 Feature Map의 채널 수임.

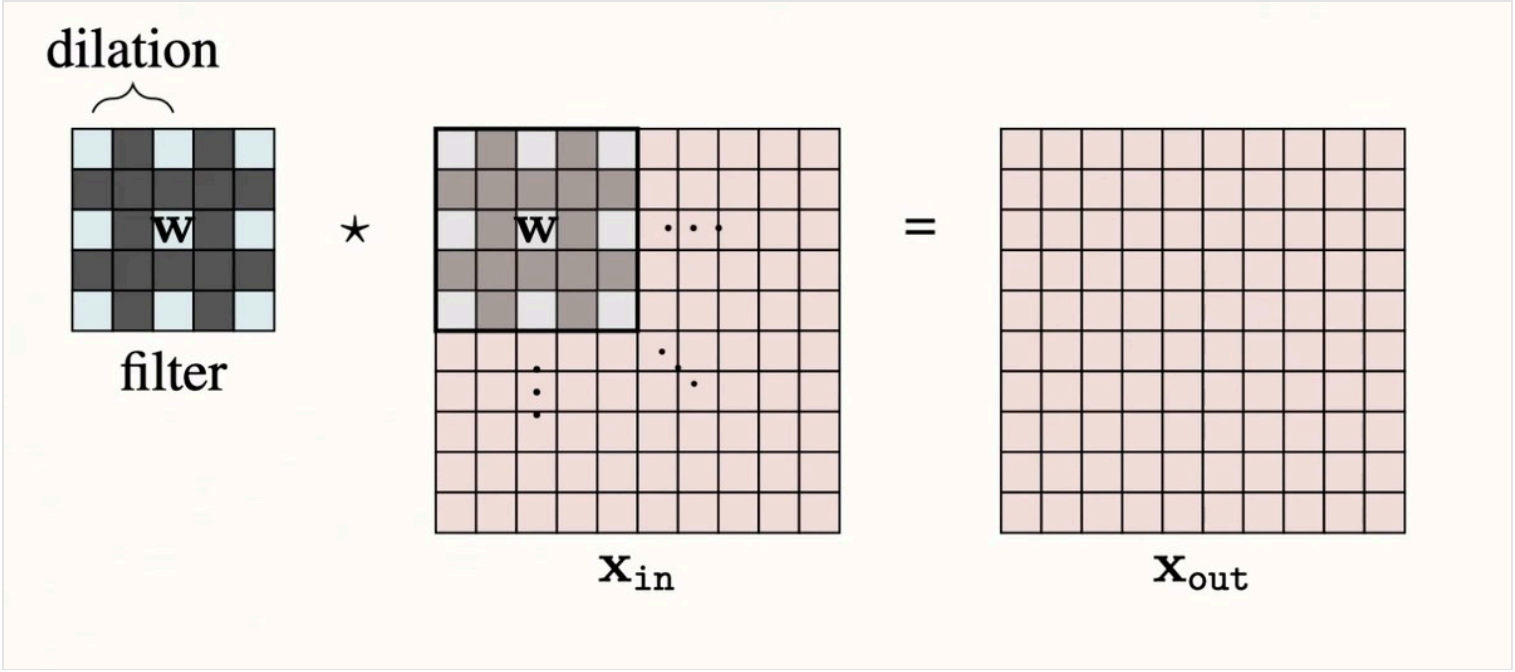
- → Convolution Filter 하나는 입력의 모든 채널을 함께 참조하여 출력 채널 하나를 생성함.
- → 여러 개의 Filter를 사용하면 여러 개의 출력 채널을 만들 수 있음.
- 중요한 점!
- → 채널 수가 3에서 1로 바뀐다고 텐서가 3차원에서 1차원으로 바뀌는 것은 아님!
- → H × W × 1도 여전히 높이·너비·채널의 3개 축을 가진 텐서임. (배치 차원 제외)
- → 일반적인 2D Convolution에서는 $H \times W \times C_{in} \rightarrow H' \times W' \times C_{out}$ 형태로, 공간 크기와 채널 수가 바뀔 수 있지만 텐서의 축 구조는 유지됨.
- 반대로 출력 이미지에 RGB 색상이 필요하다면?
- → 마지막 Convolution Layer의 출력 채널 수를 3으로 설정하면 RGB 값을 출력할 수 있음.
- EX
- → Feature Map: $H \times W \times C_{in}$
- → Convolution ($C_{out} = 3$)
- → RGB Output: $H' \times W' \times 3$
- 즉, Convolution은 1차원과 Color 사이를 변환하는 것이 아니라 입력 채널의 특징을 결합하여 원하는 개수의 출력 채널을 만드는 연산임!
- general
  → multi-input, mult-output

- pooling
- 1) max pooling
  2) mean pooling
- 출력의 안정성 강화
- 일반적인 Pooling은 채널들을 통합하는 것이 아니라, 각 채널에서 공간 방향의 정보를 요약하는 연산임!
- → Max Pooling: 각 채널의 일정 영역에서 최댓값을 선택함.
- → Average(Mean) Pooling: 각 채널의 일정 영역에서 평균값을 계산함.
- EX
- → Input: $H \times W \times C$
- → Spatial Pooling → Output: $H' \times W' \times C$
- → 높이와 너비(H, W)는 줄어들 수 있지만, 일반적인 Spatial Pooling에서는 채널 수 C는 그대로 유지됨.
- 즉, 각 채널에 대해 독립적으로 공간 정보를 요약하는 것!
- 채널 방향의 정보를 통합하거나 채널 수를 줄이는 연산은 별도로 구분해야 함.
- → EX: 1×1 Convolution을 사용하면 여러 입력 채널을 결합하여 출력 채널 수를 조절할 수 있음.
- 참고!
- → Global Average Pooling은 각 채널의 공간 전체를 평균 내어 $H \times W \times C \rightarrow 1 \times 1 \times C$로 만들 수 있음.
- → 이 경우에도 채널 수 자체는 유지됨.
- 즉, Spatial Pooling은 주로 공간 해상도를 줄이는 연산이고, 채널 방향의 결합·축소와는 다른 개념임!
- 다운 샘플링(차원 낮춤) 도 할 수 있음

- 스트라이드 연산
- conv + 풀링 + 다운샘플링

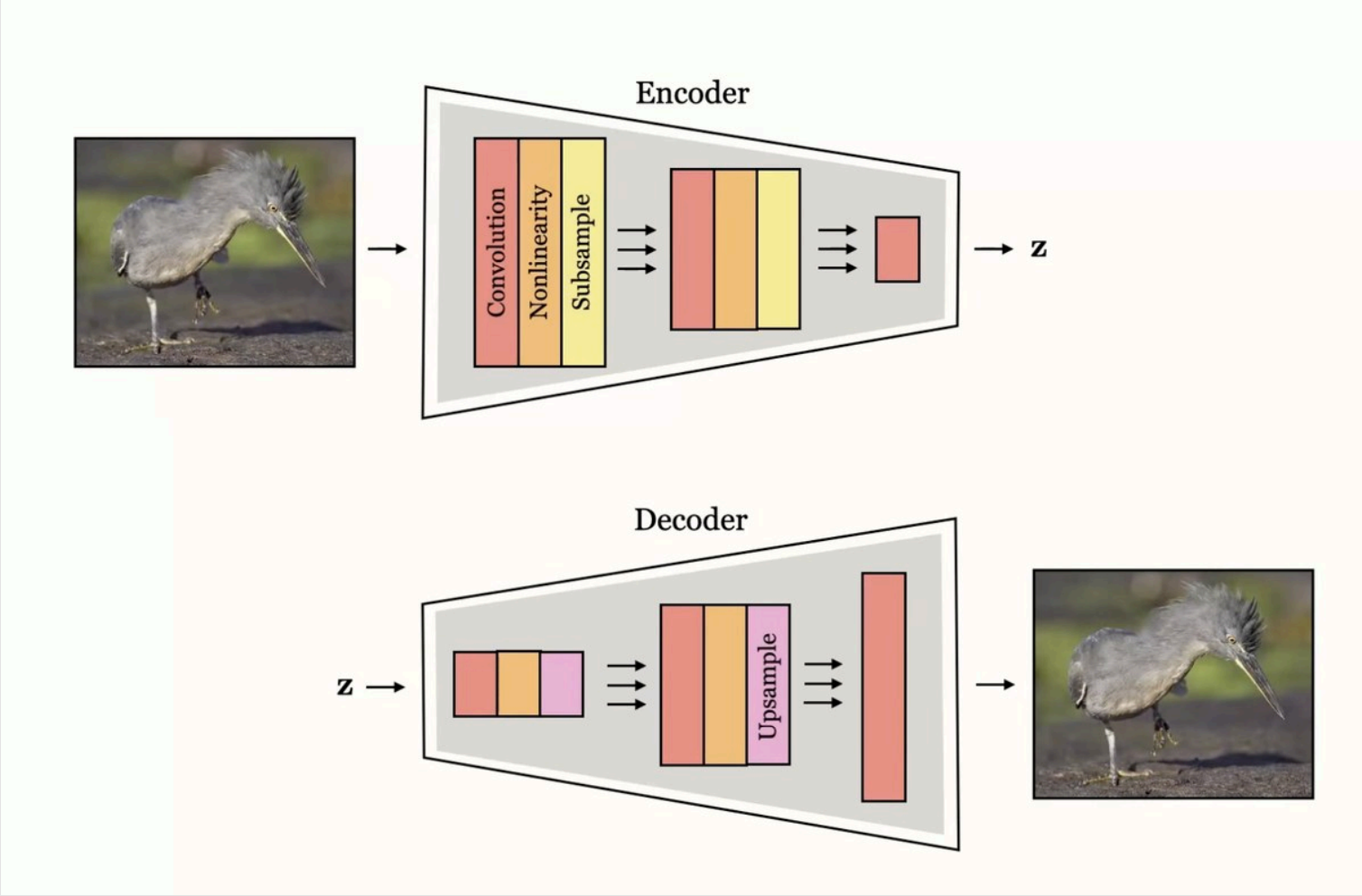


- Stride(스트라이드)
- → Convolution에서 필터를 한 번 이동할 때 몇 칸씩 움직이는지를 나타냄.
- → Stride = 1이면 필터를 한 칸씩 이동함.
- → Stride가 커지면 필터가 더 큰 간격으로 이동하므로 출력의 공간 크기가 작아짐. (다운샘플링 효과)
- 바로 앞 그림의 경우
- → 입력 크기: 11×11
- → 필터 크기: 3×3
- → Stride = 4
- → Padding = 0 (없음)
- → 출력 크기: 3×3
- 출력 크기 공식 (한 축 기준)
- Output Size = ⌊(Input Size − Filter Size + 2×Padding) / Stride⌋ + 1
- 그림에 적용하면
- ⌊(11−3)/4⌋+1 = 3
- → 가로·세로 각각 3이므로 출력은 3×3!
- 즉, 이 그림에서는 필터를 가로·세로 방향으로 4칸씩 이동하여 11×11 입력을 3×3 출력으로 줄인 것임.
- Stride를 크게 하면 출력 크기와 연산량을 줄일 수 있지만, 일부 위치의 정보를 놓칠 수도 있음.
- Dilated filter

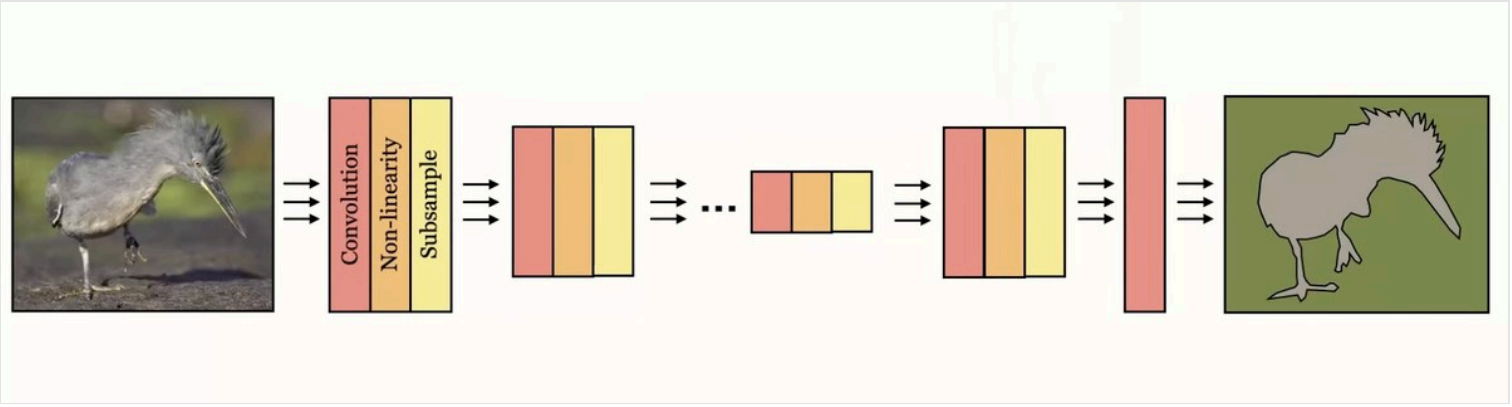


- Dilated Convolution(팽창 합성곱)
- → Convolution Filter의 원소 사이에 간격을 두고 입력에 적용하는 방식!
- → 일반적인 Convolution보다 더 넓은 영역의 정보를 참조할 수 있음.
- Dilation Rate(d)
- → Filter의 인접한 원소가 입력에서 몇 칸 간격으로 적용되는지를 나타냄.
- → d = 1: 일반적인 Convolution과 같음. (간격 없이 적용)
- → d = 2: Filter의 인접한 원소를 입력에서 2칸 간격으로 적용함.
- 그림에서는 Filter 원소 사이에 0이 삽입된 것처럼 표현할 수 있음.
- → 하지만 실제로 학습하는 가중치가 늘어나거나, 새로운 학습 파라미터를 0으로 추가하는 것은 아님!
- 유효 커널 크기(Effective Kernel Size)
- $k_{eff} = (k - 1)d + 1$
- → k: 원래 Filter의 한 축 크기
- → d: Dilation Rate
- → k_{eff}: 간격을 포함해 실제로 참조하는 영역의 한 축 크기
- EX
- → Filter Size: 3 × 3
- → Dilation Rate: d = 2
- → $k_{eff} = (3 - 1) \times 2 + 1 = 5$
- → 즉, 3 × 3 Filter가 공간적으로는 5 × 5 영역에 걸쳐 적용되는 효과를 얻음!
- 하지만 학습하는 Filter의 가중치 수는 여전히 3 × 3 = 9개임. (입출력 채널 한 쌍 기준)
- 장점
- → 학습 파라미터 수를 늘리지 않고 더 넓은 수용 영역(Receptive Field)을 확보할 수 있음.
- → 따라서 더 넓은 Context 정보를 활용할 수 있음.
- 즉, Dilated Convolution은 Filter 원소 사이의 간격을 넓혀 파라미터 수는 유지하면서 수용 영역을 확장하는 방법임!

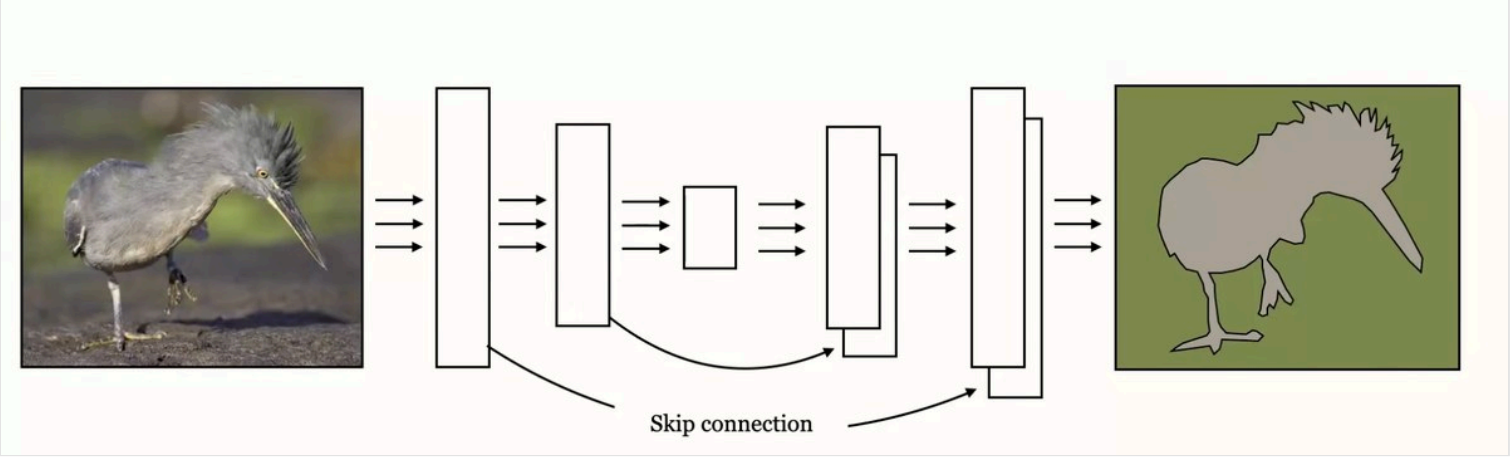
- popular CNN Architectures
- Encoder-Decoder Architectures



- Encoder: 원래 이미지 → 낮은 차원의 Z (다운샘플링)
  Decoder: Z → 이미지 예측 (업샘플링)
- Decoder의 출력이 반드시 Encoder의 입력과 일치해야 하는 것은 아님!
- Autoencoder의 경우
- → Encoder: 입력 x → 잠재 표현 z
- → Decoder: z → 복원된 입력 $\hat{x}$
- → 목표: $\hat{x}$가 원래 입력 x와 최대한 비슷해지도록 학습함. (입력 재구성)
- 하지만 일반적인 Encoder–Decoder 구조에서는 입력과 다른 출력을 만드는 것도 가능함!
- EX
- → Segmentation: 입력 이미지 → 픽셀별 클래스 예측
- → Image-to-Image Translation: 입력 이미지 → 변환된 이미지
- 즉, 입력 재구성은 Autoencoder의 목표이고, Encoder–Decoder 전체에 공통인 목표는 아님!
- Decoder는 이미지를 만들어 낼 수도 있음
- Image-to-image

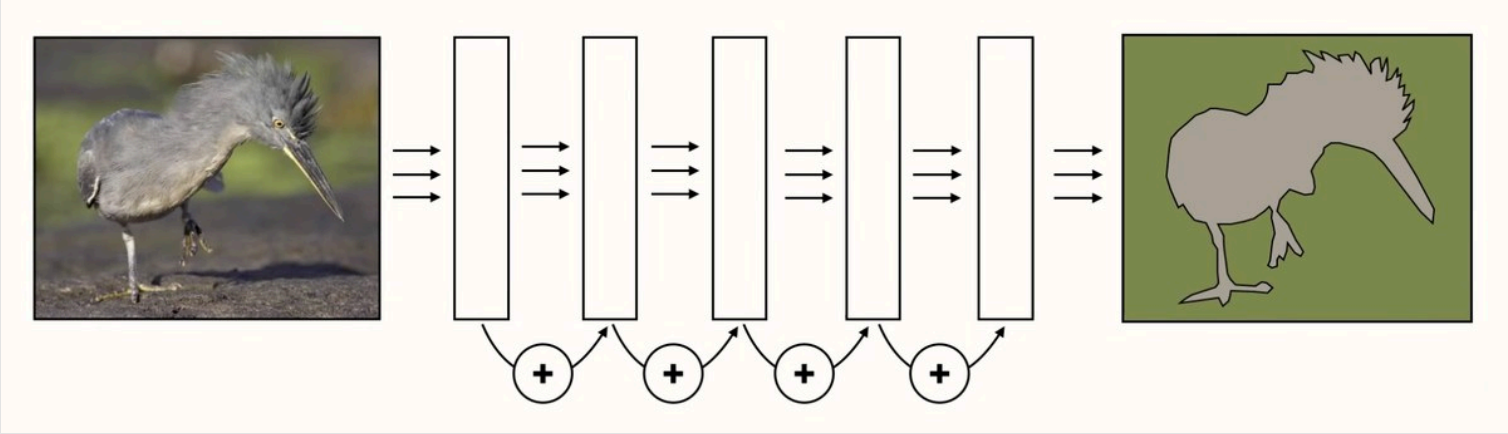


- segmentation도 가능!
- (before)차원을 줄이지 않음.(고해상도 유지 → 비용이 커짐)
- after → U-net



- Skip connection이라는 연결이 있음.
  중간 layer를 건너 뛰는 형태
- U-Net의 Skip Connection은 Encoder의 특징 정보를 Decoder로 직접 전달하는 연결임!
- → Encoder에서 추출한 Feature Map을 공간 해상도가 대응되는 Decoder 단계에 연결함. (동일한 Spatial Scale)
- → 일반적으로 Encoder의 Feature Map과 Decoder에서 업샘플링한 Feature Map을 채널 방향으로 연접(Concatenation)함.
- → 이후 Convolution을 적용해 두 특징을 결합함.
- 왜 이렇게 할까?
- Encoder의 깊은 층
- → 다운샘플링을 거치면서 공간 해상도가 낮아짐.
- → 더 넓은 문맥(Context)과 의미적 특징(Semantic Features)을 학습하는 데 유리함.
- Encoder의 얕은 층
- → 상대적으로 높은 공간 해상도를 유지함.
- → 객체의 경계, 위치, 작은 구조 같은 세부 정보를 더 잘 보존할 수 있음.
- 하지만 Encoder에서 다운샘플링을 반복하면 세부적인 공간 정보가 손실될 수 있음!
- → Skip Connection으로 얕은 층의 고해상도 특징을 Decoder에 직접 전달함.
- → Decoder는 깊은 층의 의미적 특징과 Encoder에서 전달된 세부 정보를 함께 활용할 수 있음.

- 즉, U-Net은 낮은 공간 해상도에서 얻은 넓은 문맥·의미적 특징과 높은 공간 해상도에서 보존된 세부 정보를 결합하는 구조!
- → 특히 Segmentation처럼 픽셀 단위로 정확한 위치와 경계를 예측해야 하는 작업에 유리함.
- ResNet



- ResNet은 잔차 블록(Residual Block) 단위로 Skip Connection을 사용함.
- 입력 x를 두 경로로 전달함.
- → Residual Path: F(x)를 학습함.
- → Skip Path: 입력 x를 그대로 전달함. (Identity Mapping)
- 두 경로의 출력을 더함!
- y = F(x) + x (입출력 차원이 같은 경우)
- 차원이 다르면 Skip Path에 Projection 등을 적용해 차원을 맞출 수 있음.
- 이때 처리할지/스킵할지 선택하는 게 아님!
- → 두 경로는 항상 합산됨. (별도의 게이팅·선택 구조가 아님)
- 장점
- → Skip Path를 통해 기울기가 더 잘 전달되어 깊은 네트워크의 학습이 쉬워짐.
- → F(x)=0에 가까워지면 항등함수 y≈x를 쉽게 표현할 수 있음.
- 즉, 최적의 깊이를 자동으로 선택하는 구조가 아니라 잔차 학습으로 깊은 네트워크의 최적화를 돕는 구조!

- 차원 추가
- Convolution은 기본적으로 평행이동 등변성(Translation Equivariance)을 가짐.
- → 입력 이미지가 이동하면 출력 특징 맵도 그에 따라 이동함. (stride=1, 경계 효과 등을 제외한 경우)
- Translation Invariance(평행이동 불변성)와는 다름!
- → Equivariance: 입력이 이동하면 출력도 대응해서 이동함.
- → Invariance: 입력이 이동해도 출력이 같게 유지됨.
- Pooling 등을 사용하면 작은 위치 변화에 대해 근사적인 불변성을 얻을 수 있음. (항상 보장되는 것은 아님)
- 주의할 점!
- → Stride나 Pooling으로 다운샘플링할 때 Aliasing이 발생할 수 있음.
- → 이 때문에 작은 위치 이동에도 출력이 달라져 등변성이나 기대한 불변성이 깨질 수 있음.
- 위치 정보를 명시적으로 사용하고 싶다면?

- 1. MLP 사용
- → 일반적인 MLP는 평행이동 불변성이나 등변성을 구조적으로 강제하지 않음.
- → 대신 CNN의 지역성·가중치 공유라는 귀납적 편향도 포기하는 선택임.
- 2. Positional Encoding 사용
- → 입력 특징에 위치 정보를 추가하여 모델이 위치를 구분할 수 있게 함.
- → 위치 인코딩은 학습 가능한 형태(Learned)도 있고, 고정된 형태(Fixed)도 있음.
- → Sinusoidal Positional Encoding은 학습하지 않는 고정형의 대표적인 예!
- 즉, Positional Encoding은 위치를 반드시 추가로 학습하는 방식이 아니라, 위치 정보를 모델에 제공하는 장치임.
- Neural Fields
- → 신경망을 이용하여 공간이나 시간의 각 좌표에 대응하는 값을 표현하는 방법!
- → 좌표를 입력받으면 해당 위치의 신호 값을 출력하는 연속 함수(Continuous Function)를 학습함.
- 즉, 데이터를 고정된 Grid의 값으로만 표현하는 대신, 신경망 자체가 좌표 → 값의 함수를 표현하도록 만드는 것!
- 수식으로 표현하면
- $f_\theta(p) \rightarrow$ value
- → p: 입력 좌표 (공간 좌표 또는 시간 좌표 등)
- → θ: 신경망의 학습 가능한 파라미터
- → value: 해당 좌표에서 예측한 신호 값
- 기존 Grid-based Representation과의 차이는?
- Grid-based Representation
- → 이미지의 Pixel이나 3D 공간의 Voxel처럼 미리 정해진 격자 위치에 값을 저장하거나 계산함.
- Neural Fields
- → 좌표를 입력하면 신경망이 그 위치의 값을 예측함.
- → 따라서 학습한 영역 안에서 격자점 사이의 임의 좌표도 질의(Query)할 수 있음.
- → 단, 임의 좌표에서 값을 출력할 수 있다는 것이 원본에 없는 고주파 세부 정보를 반드시 정확하게 복원한다는 의미는 아님!
- 앞에서 본 SIREN과 연결하면?
- → SIREN은 Sin 활성화 함수를 사용하는 신경망으로, Neural Field를 구현하는 방법 중 하나임.
- → 좌표를 입력받아 이미지·오디오·3D 형상 등의 연속 신호를 학습한다는 점에서 Neural Fields의 대표적인 사례로 볼 수 있음.
- → 하지만 모든 Neural Field가 SIREN을 사용하는 것은 아님!
- → 일반적인 MLP나 Positional Encoding 등을 활용하는 방식도 있음.
- 즉, Neural Fields는 좌표에 따른 신호를 신경망이 표현하는 방식이고, SIREN은 이를 구현하는 구체적인 네트워크 구조 중 하나임!